

Investigación Técnica: Sistemas de Codificación de Caracteres y Estructuras de Almacenamiento de Datos

Estudiante: Diego Rodríguez
Matrícula: 2026-0259
Materia: Programación para Mecatrónicos
Profesor: Carlos Antonio Pichardo Viuque
Fecha de Entrega: 26 de junio de 2026

Introducción: El flujo de la información en el hardware y el software

Cuando nos adentramos en el diseño mecatrónico, tendemos a pensar inmediatamente en variables físicas: el torque de un motor paso a paso, la corriente que circula por un puente H, o el voltaje analógico que entrega un sensor ultrasónico. Sin embargo, un sistema electrónico moderno no está aislado. Para que un brazo robótico responda a una orden, o para que un sistema automatizado registre su productividad, los datos deben transformarse, empaquetarse y transmitirse.

Las computadoras y los microcontroladores (como un procesador ARM o un microchip PIC) habitan en un ecosistema puramente binario de unos y ceros. Ellos no tienen un concepto nativo de lo que es la letra "A", ni entienden de forma espontánea que un conjunto de números representa la lectura crítica de la temperatura de un caldero industrial.

Para resolver esta brecha entre el entendimiento humano y la lógica digital, la ingeniería de software desarrolló dos herramientas fundamentales: las **codificaciones de caracteres**, que definen los diccionarios matemáticos para traducir letras a bytes, y los **formatos de archivos**, que dictan las reglas gramaticales para organizar esos bytes en estructuras con significado técnico.

Esta investigación desglosa de manera cercana y detallada cómo funcionan estos estándares y por qué cambian las reglas del juego según el hardware que utilicemos.

Nota de corrección técnica: En los temas planteados originalmente, se incluyeron las siglas "UTF18" y "CVS". Haciendo una revisión de la documentación internacional, se identifica que corresponden a errores comunes de transcripción de los estándares oficiales **UTF-16** (Unicode de 16 bits) y **CSV** (Comma-Separated Values). Para garantizar el rigor técnico del reporte, la investigación abordará las tecnologías bajo sus nombres normativos correctos.

1. Codificación de Caracteres: ¿Cómo leen las máquinas?

Codificar es, en esencia, asignar un número entero exclusivo a cada símbolo de nuestro lenguaje. Si dos sistemas no se ponen de acuerdo en el diccionario de codificación que están usando, la comunicación se rompe por completo, dando lugar a errores de lectura o comandos corruptos que en entornos industriales pueden descalibrar un actuador o malinterpretar un límite de carrera.

1.1 ASCII: El origen minimalista de la comunicación serial

El estándar ASCII (American Standard Code for Information Interchange) nació en los albores de la computación comercial en la década de 1960. Diseñado con una mentalidad de ahorro estricto de recursos, ASCII trabaja con un sistema de **7 bits**, lo que provee exactamente 128 combinaciones posibles (del 0 al 127 en formato decimal).

Estas 128 posiciones se distribuyen de forma muy inteligente: las primeras 32 posiciones (0 al 31) no corresponden a letras visibles, sino a "caracteres de control" heredados de la época de los teletipos, como el salto de línea (LF, decimal 10) y el retorno de carro (CR, decimal 13). Las posiciones restantes contienen el alfabeto en inglés (mayúsculas y minúsculas), los números del 0 al 9 y símbolos de puntuación básicos.

Conexión Mecatrónica con ASCII:

En la comunicación industrial serial (como los protocolos RS-232, RS-485 o Modbus ASCII), cada byte transmitido cuenta. Si estás usando un microcontrolador de recursos limitados para enviar datos de telemetría rápidos, ASCII es perfecto porque ocupa exactamente 1 byte por carácter (dejando el octavo bit en cero). Mandar el comando string "START" consume solo 5 bytes de memoria y es interpretado inmediatamente por el procesador sin necesidad de librerías pesadas de decodificación. Su talón de Aquiles es su limitación geográfica: no puede representar tildes, la letra "ñ", ni caracteres de otros alfabetos.

1.2 UTF-8: El estándar universal inteligente

A medida que la informática se expandió por el mundo, la limitación de 128 caracteres de ASCII se volvió insostenible. Así nació el proyecto Unicode, un esfuerzo por mapear absolutamente todos los caracteres de todos los idiomas del planeta. Para implementar Unicode sin destruir el software existente, se creó **UTF-8**.

UTF-8 es una obra maestra de la optimización porque utiliza un **ancho variable de bytes (de 1 a 4 bytes)**. Su funcionamiento sigue las siguientes reglas empíricas:

- Si el carácter pertenece al inglés (estándar ASCII), UTF-8 usa únicamente **1 byte**. Esto significa que UTF-8 es 100% compatible hacia atrás con ASCII. Un archivo ASCII viejo es un archivo UTF-8 perfecto.
- Si necesitas usar caracteres latinos extendidos (como una "ó" con tilde o una "ñ"), el sistema detecta un bit de prefijo y se expande automáticamente a **2 bytes**.
- Para caracteres más complejos como simbología matemática avanzada, alfabetos cirílicos o incluso emojis de advertencia de seguridad, llega a utilizar **3 o 4 bytes**.

Actualmente, UTF-8 es el dueño indiscutible del desarrollo web, sistemas Linux, scripts de automatización en Python y bases de datos integradas, convirtiéndose en el estándar por defecto si tu código interactúa con interfaces de usuario modernas o servicios web en la nube.

1.3 UTF-16: La alternativa de base ancha

A diferencia del enfoque flexible desde un byte de UTF-8, **UTF-16** toma una ruta diferente: utiliza bloques base de **16 bits (2 bytes)** para representar la información, pudiendo expandirse a 4 bytes mediante el uso de pares sustitutos para símbolos extremadamente raros.

Esto significa que para almacenar una cadena de texto en español o inglés, UTF-16 consumirá exactamente el doble de espacio de almacenamiento en comparación con UTF-8 o ASCII, ya que cada letra básica ocupará obligatoriamente 2 bytes completos en lugar de uno.

¿Dónde se justifica su uso? Se vuelve muy eficiente en regiones asiáticas (China, Japón, Corea), puesto que la enorme mayoría de sus ideogramas complejos caben directamente en el bloque inicial de 2 bytes, mientras que en UTF-8 requerirían 3 o 4 bytes. En el software, es el núcleo de procesamiento interno del sistema operativo Microsoft Windows y de la Máquina Virtual de Java (JVM). En el mundo embebido, sin embargo, se evita debido a su propensión a desperdiciar memoria y a la complejidad añadida de gestionar el "Endianness" (el orden en que los procesadores leen los bytes más significativos).

2. Formatos de Archivos: ¿Cómo organizamos los datos?

Una vez que resolvemos cómo codificar letras individuales, nos topamos con el siguiente problema de ingeniería: la estructura. Si un sensor envía una ráfaga de texto que dice "24580", los datos son inútiles si el programa receptor no sabe si "24" corresponde a la temperatura y "580" a las revoluciones por minuto de un motor. Los formatos de archivos nos dan las reglas para empaquetar estos conjuntos de datos de forma predecible.

2.1 JSON: Estructuras dinámicas orientadas a la nube y el IoT

JSON (JavaScript Object Notation) es un formato de texto plano que se ha convertido en el estándar de facto para el Internet de las Cosas (IoT). Su filosofía se basa en un concepto muy intuitivo para los desarrolladores: colecciones de parejas de **Clave / Valor** y arreglos ordenados de datos.

Su sintaxis es minimalista, limpia y elimina cualquier adorno innecesario, utilizando llaves {} para delimitar objetos y corchetes [] para listas indexadas.

Ejemplo práctico de una trama JSON de telemetría:

```
{
  "id_estacion": "NODO-MECATRONICO-04",
  "tiempo_operacion_min": 120,
  "lecturas_sensores": [
    {"tipo": "presion", "valor": 4.2},
    {"tipo": "caudal", "valor": 15.8}
  ],
  "alerta_critica": false
}
```

En programación para mecatrónicos, JSON es el lenguaje que hablan los protocolos de red modernos como **MQTT** o HTTP REST. Si estás enviando los estados de variables de un PLC hacia una base de datos en la nube (como Firebase o AWS IoT Core) para construir un tablero de control (dashboard), empaquetar los datos en JSON permite que cualquier lenguaje moderno (Python, JavaScript, C#) los procese con apenas una línea de código.

2.2 XML: Rigidez estructural para entornos industriales y normativos

XML (eXtensible Markup Language) es un lenguaje de marcado jerárquico basado en el uso de etiquetas personalizadas encerradas por símbolos de menor y mayor que (<etiqueta>...</etiqueta>). A diferencia de JSON, XML es sumamente estricto y formal; exige de manera

obligatoria etiquetas de cierre, respeta de forma estricta las mayúsculas y minúsculas, y permite vincular archivos de esquema (XSD) para validar la integridad del documento antes de abrirlo.

Ejemplo de la misma estación de telemetría representada en XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<estacion id="NODO-MECATRONICO-04">
  <tiempo_operacion_min>120</tiempo_operacion_min>
  <lecturas_sensores>
    <sensor tipo="presion">4.2</sensor>
    <sensor tipo="caudal">15.8</sensor>
  </lecturas_sensores>
  <alerta_critica>false</alerta_critica>
</estacion>
```

La importancia de XML en la automatización industrial:

Aunque muchos programadores jóvenes consideran que XML es demasiado "pesado" y redundante (debido a la cantidad de bytes que gasta repitiendo etiquetas), en la ingeniería industrial tiene un peso enorme. El estándar internacional **IEC 61131-3**, que regula la programación de controladores lógicos programables (PLC), utiliza formatos basados en XML (como PLCopen XML) para permitir el intercambio de lógica de control (diagramas de escalera, bloques funcionales) entre plataformas de desarrollo de distintos fabricantes como Siemens, Schneider Electric o CODESYS. Su verbosidad es el precio a pagar por una robustez y una seguridad incomparables.

2.3 CSV: El rey indiscutible de la adquisición masiva de datos (Data Logging)

El formato CSV (Comma-Separated Values) representa el enfoque más crudo y directo de almacenamiento tabular. No existen jerarquías complejas, ni objetos anidados, ni etiquetas de seguridad. Un archivo CSV es, literalmente, una matriz o tabla de texto donde cada línea representa una fila de datos y cada columna está separada por un carácter delimitador (típicamente una coma o un punto y coma).

Ejemplo de registro continuo en formato CSV:

```
timestamp,voltaje_bateria,corriente_motor,temperatura_mosfet
2026-06-26T01:00:00Z,12.42,1.15,35.4
2026-06-26T01:00:01Z,12.39,1.22,35.8
2026-06-26T01:00:02Z,12.35,1.54,36.2
```

¿Por qué CSV es indispensable para un Ingeniero Mecatrónico?

Imagina que estás haciendo pruebas de fatiga mecánica en un prototipo o analizando las

vibraciones de un motor a altas velocidades de muestreo. El hardware embebido necesita tomar miles de muestras por segundo y guardarlas en tiempo real. Escribir estructuras complejas en JSON o XML consumiría valiosos ciclos de reloj del procesador y llenaría la memoria en segundos con puro texto repetitivo.

Con CSV, el microcontrolador simplemente concatena el valor numérico y una coma, y lo vuelca de forma secuencial en una tarjeta de memoria MicroSD a través del bus SPI. Posteriormente, este archivo plano puede ser importado de forma directa en potentes entornos de software de análisis de ingeniería como **MATLAB, GNU Octave, Python (con la librería Pandas) o Microsoft Excel** para procesar filtros matemáticos, calcular promedios, derivar variables y generar gráficas de control de manera inmediata.

3. Matriz de Criterios de Selección para Ingeniería

Para un ingeniero mecatrónico, la ingeniería consiste en saber tomar decisiones de diseño basadas en restricciones técnicas (memoria, ancho de banda, capacidad de cómputo). La siguiente tabla consolida los criterios clave para seleccionar los formatos estudiados:

Formato	Complejidad de Cómputo	Consumo de Ancho de Banda	Flexibilidad Estructural	Aplicación Predominante en Mecatrónica
JSON	Baja-Media (Fácil de parsear en C/Python)	Eficiente (Estructura compacta)	Alta (Soporta objetos jerárquicos)	Mensajería IoT (MQTT), APIs de control web, configuraciones de software robótico.
XML	Alta (Requiere parsers robustos tipo DOM/SAX)	Alto (Estructura redundante por etiquetas)	Máxima (Permite esquemas formales XSD)	Intercambio de programas de PLC (IEC 61131-3), archivos de configuración industrial robustos.
CSV	Mínima (Cualquier bucle simple lee comas)	Mínimo (Almacena solo datos puros)	Nula (Solo estructuras bidimensionales planas)	Data Logging de sensores a alta velocidad, almacenamiento en tarjetas SD, análisis en MATLAB/Excel.

Conclusión y Reflexión Académica

A lo largo de esta investigación, queda en evidencia que el desarrollo de soluciones mecatrónicas modernas va mucho más allá del ensamblaje mecánico o el diseño de circuitos impresos. En la era de la industria 4.0 y los sistemas ciberfísicos, los flujos de información digital imponen sus propias leyes físicas. Un código de control optimizado a la perfección puede fallar estrepitosamente si el formato en el que transmite sus alertas corrompe los caracteres especiales, o si satura el bus de comunicación de un sistema embebido al elegir un formato de archivo excesivamente pesado de manera innecesaria.

Como estudiantes de programación aplicada a la ingeniería, comprender el balance técnico detrás de estas tecnologías nos permite diseñar de forma balanceada: seleccionando diccionarios ligeros como ASCII y almacenamiento plano como CSV cuando los recursos de hardware sean escasos y la velocidad sea la prioridad absoluta; y migrando hacia la flexibilidad universal de UTF-8 y la elegancia conectiva de JSON cuando el sistema deba integrarse con redes de computación de alto nivel y servidores en la nube. Dominar la manipulación, traducción e interpretación de estas estructuras de software es lo que consolida el perfil interdisciplinario y estratégico del ingeniero mecatrónico actual.